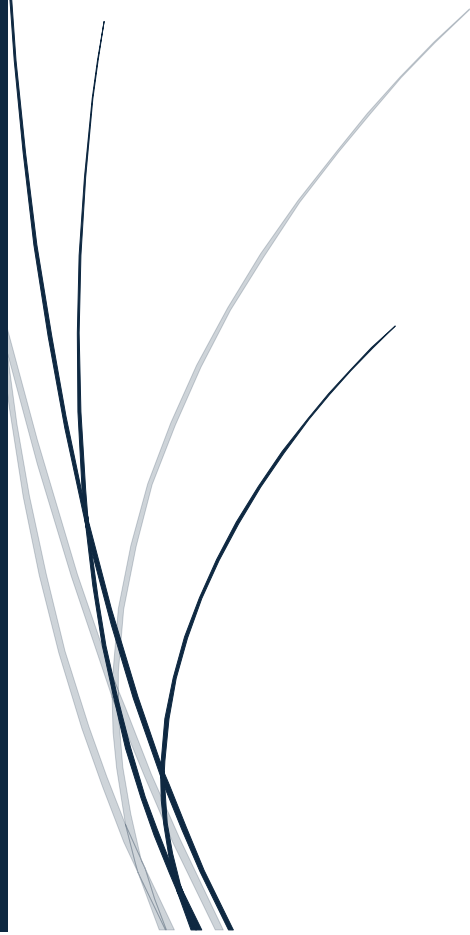




۱۲/۱۹/۲۰۲۰

Unified Feedback & Ticketing Management System **(UFTMS)**



Amirreza Tabeshfard

فهرست

۲.....	Unified Feedback & Ticketing Management System (UFTMS)
۲.....	موضوع پروژه (شرح جامع، اصولی و بازنگری شده) 
۳.....	احراز هویت، مجوزدهی و کنترل دسترسی 
۴.....	فرآیند مدیریت تیکت و گردش اطلاعات 
۵.....	ذخیره سازی داده ها بدون استفاده از Database 
۶.....	اهداف معماری و آموزشی پروژه 
۷.....	طراحی معماری لایه ای Web API 
۹.....	مدیریت تیکت و گردش کار 
۱۰.....	مدل های دامنه (Domain Models) 
۱۱.....	DTO ها (Data Transfer Objects) 

Unified Feedback & Ticketing Management System (UFTMS)

🔗 موضوع پروژه (شرح جامع، اصولی و بازنگری شده)

این پروژه با هدف طراحی و پیاده‌سازی یک سامانه یکپارچه مدیریت تیکت و بازخورد کاربران توسعه داده می‌شود که به سازمان‌ها و سامانه‌های نرم‌افزاری این امکان را می‌دهد تا تمامی مشکلات فنی، نارسایی‌های عملکردی، انتقادات، پیشنهادات و درخواست‌های بهبود کاربران را به‌صورت ساخت‌یافته، قابل پیگیری و مبتنی بر فرآیند مدیریت کنند.

هسته اصلی این سامانه به‌صورت یک Backend مستقل و مبتنی بر ASP.NET Core Web API با استفاده از NET 8. پیاده‌سازی می‌شود و تمامی منطق تجاری (Business Logic)، قوانین امنیتی، مدیریت کاربران و گردش کار تیکت‌ها در این لایه متمرکز خواهد بود. این Backend به‌صورت کامل از لایه نمایش جدا بوده و از طریق API‌های استاندارد RESTful در اختیار کلاینت‌ها قرار می‌گیرد.

در لایه رابط کاربری، Front-End سامانه به‌صورت کامل با AngularJS توسعه داده می‌شود. AngularJS به‌عنوان مصرف‌کننده‌ی Web API عمل کرده و مسئول نمایش داده‌ها، مدیریت تعاملات کاربر، ارسال درخواست‌ها به Backend و پیاده‌سازی تجربه کاربری خواهد بود. این جداسازی کامل Front-End و Back-End باعث می‌شود سیستم از نظر معماری ماژولار، قابل نگهداری و قابل توسعه باشد.

🔒 احراز هویت، مجوزدهی و کنترل دسترسی

سامانه به صورت کامل از Authentication و Authorization پشتیبانی می کند. تمامی کاربران برای دسترسی به امکانات سیستم باید احراز هویت شوند و پس از آن، سطح دسترسی آن ها بر اساس نقش های تعریف شده کنترل می شود. نقش هایی مانند:

- کاربر عادی (ثبت و پیگیری تیکت)
- کارشناس پشتیبانی (بررسی و پاسخ دهی)
- مدیر سیستم (مدیریت کاربران و نظارت کلی)

در نظر گرفته می شود. این مکانیزم امنیتی در لایه Web API پیاده سازی شده و AngularJS تنها به عنوان مصرف کننده ی امن این سرویس ها عمل می کند، بدون آن که منطق امنیتی در Front-End قرار گیرد.

فرآیند مدیریت تیکت و گردش اطلاعات

کاربران پس از ورود به سامانه قادر خواهند بود تیکت‌هایی با موضوعات مختلف ثبت کنند. هر تیکت دارای ساختاری مشخص و قابل ردیابی است که شامل:

- شناسه یکتا
- عنوان و توضیحات تفصیلی
- نوع تیکت (مشکل، انتقاد، پیشنهاد، درخواست بهبود)
- سطح اولویت
- وضعیت پردازش
- تاریخچه تغییرات و پاسخ‌ها

تمامی مراحل رسیدگی به تیکت‌ها به صورت شفاف ثبت می‌شود تا امکان پیگیری دقیق، تحلیل عملکرد و استخراج گزارش‌های مدیریتی فراهم گردد.

ذخیره‌سازی داده‌ها بدون استفاده از Database

یکی از ویژگی‌های متمایز و آگاهانه‌ی این پروژه، عدم استفاده از هرگونه پایگاه داده سنتی است. به‌جای آن، اطلاعات سامانه به‌صورت فایل‌محور و از طریق **Serialization** ذخیره و بازیابی می‌شوند.

در این پروژه از دو روش ذخیره‌سازی داده استفاده می‌شود:

▪ JSON Serialization

○ برای داده‌هایی که نیاز به خوانایی انسانی، توسعه‌پذیری و امکان بررسی و تحلیل دارند؛ مانند تیکت‌ها، پیام‌ها و گزارش‌ها.

▪ Binary Serialization

○ برای داده‌هایی که حجم کمتر، سرعت بیشتر یا سطحی از پنهان‌سازی ساختار داده اهمیت دارد؛ مانند اطلاعات حساس‌تر یا ساختارهای داخلی سیستم.

انتخاب نوع **Serialization** برای هر نوع داده بر اساس اهمیت، حساسیت، کارایی و نیاز به خوانایی انجام می‌شود و این تصمیم‌ها بخشی از طراحی معماری پروژه محسوب می‌شوند.

🎯 اهداف معماری و آموزشی پروژه

این پروژه با رویکردی فراتر از یک پیاده‌سازی ساده طراحی شده و اهداف زیر را دنبال می‌کند:

- طراحی و پیاده‌سازی Web API استاندارد در .NET 8.
- پیاده‌سازی امنیت بدون وابستگی به Database
 - تجربه امنیت بدون Database
- استفاده عملی از AngularJS در نقش Front-End
 - استفاده عملی AngularJS
- تجربه معماری فایل محور به جای پایگاه داده
 - تمرین معماری لایه‌ای واقعی
 - بررسی معماری فایل محور به جای دیتابیس
- بررسی عملی تفاوت‌ها و کاربردهای JSON و Binary Serialization
 - بررسی معماری فایل محور به جای دیتابیس
- ایجاد یک سیستم قابل توسعه و قابل نگهداری

در نهایت، این سامانه نه تنها یک ابزار مدیریت تیکت، بلکه یک نمونه معماری متفاوت و قابل تحلیل خواهد بود که نشان می‌دهد چگونه می‌توان با جداسازی مسئولیت‌ها، رعایت اصول امنیتی و انتخاب آگاهانه فناوری‌ها، یک سیستم منسجم و حرفه‌ای طراحی کرد.

طراحی معماری لایه‌ای Web API

Backend پروژه بر اساس معماری لایه‌ای (Layered Architecture) طراحی می‌شود تا اصل جداسازی مسئولیت‌ها (Separation of Concerns) به‌صورت کامل رعایت شود. این معماری به‌صورت مفهومی شامل لایه‌های زیر است:

۱) Presentation Layer (API Layer)

این لایه شامل Controllerها در ASP.NET Core Web API است و تنها مسئول:

- دریافت درخواست‌ها از AngularJS
 - اعتبارسنجی اولیه ورودی‌ها
 - فراخوانی لایه Application
 - بازگرداندن پاسخ استاندارد (HTTP Response)
- هیچ منطق تجاری در این لایه قرار نمی‌گیرد.

۲) Application Layer

این لایه مغز رفتاری سیستم است و شامل:

- پیاده‌سازی Use Caseها
- مدیریت جریان کاری تیکت‌ها
- اعمال قوانین تجاری
- کنترل مجوزهای دسترسی بر اساس نقش کاربر

Application Layer مستقل از نحوه ذخیره‌سازی داده‌ها طراحی می‌شود و تنها با Interfaceها کار می‌کند.

Domain Layer (۳)

این لایه شامل مدل‌های دامنه (Domain Models / Entities) است که مفاهیم اصلی سیستم را نمایش می‌دهند. این مدل‌ها بیانگر واقعیت‌های تجاری سیستم هستند و هیچ وابستگی به Web، Serialization یا Infrastructure ندارند.

نمونه مفاهیم دامنه:

- User
- Ticket
- Message
- Status
- Priority
- Role

Domain Layer قلب مفهومی سیستم است و باید ساده، شفاف و پایدار باشد.

Infrastructure Layer (۴)

این لایه مسئول پیاده‌سازی جزئیات فنی است، از جمله:

- ذخیره‌سازی داده‌ها
- Serialization و Deserialization
- مدیریت فایل‌ها
- پیاده‌سازی Repositoryها

Infrastructure Layer پیاده‌کننده Interface‌هایی است که در Application Layer تعریف شده‌اند.

📌 مدیریت تیکت و گردش کار

کاربران احراز هویت شده می توانند تیکت هایی با موضوعات مختلف ثبت کنند. هر تیکت شامل:

- شناسه یکتا
- عنوان و توضیحات
- نوع تیکت
- سطح اولویت
- وضعیت پردازش
- تاریخچه پیام ها و پاسخ ها

وضعیت تیکت ها در طول زمان تغییر می کند و تمامی تغییرات قابل ردیابی و تحلیل هستند.

مدل‌های دامنه (Domain Models)

مدل‌های دامنه نمایش‌دهنده‌ی مفاهیم واقعی سیستم هستند. برای مثال:

- User: نمایش‌دهنده کاربر سیستم
- Ticket: موجودیت اصلی تیکت
 - TicketMessage: پیام‌ها و پاسخ‌ها
- Role: نقش کاربر
- Enums: برای وضعیت‌ها و اولویت‌ها

این مدل‌ها مستقل از نحوه انتقال داده و ذخیره‌سازی طراحی می‌شوند.

DTO ها (Data Transfer Objects)

برای ارتباط بین Web API و AngularJS از DTO ها استفاده می شود. DTO ها:

- داده ها را ساده و کنترل شده منتقل می کنند
- از افشای ساختار داخلی Domain جلوگیری می کنند
- امکان Versioning API را فراهم می کنند

انواع DTO ها شامل:

- Request DTO (برای دریافت داده)
- Response DTO (برای ارسال داده)
- Auth DTO (برای ورود و احراز هویت)